

SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History

Zhiwei Li* Yong Hu

WeChat, Tencent Inc., China

zhiweili.jay@foxmail.com rightyonghu@tencent.com

Skills: <https://github.com/Tencent/SkillHone>

Project Page: <https://zwlijay.github.io/SkillHone-Project>

Abstract

Agent skills extend language-model agents with task-specific procedures, scripts, and references, but the tasks and environments they target continually change. Existing methods improve skills in bounded runs and retain only the final artifact, discarding the decision history that later agents need to interpret prior revisions, evaluations, and rejected alternatives. We introduce SkillHone, a harness for continual agent skill evolution grounded in persistent decision history. SkillHone pairs skill revisions with evaluation-side evidence that supplies practice feedback, recording structured histories of diagnoses, revisions, evidence, and outcomes. Role-separated subagents run candidate skills on practice probes with redacted reporting and propose revisions informed by prior decisions, enabling cross-session refinement without rediscovering past rationale. On deep-research benchmarks, SkillHone runs without a pre-integrated search stack and outperforms the commercially backed deep-research agent by 15.8 points on GAIA and 3.2 points on WebWalkerQA-EN, while also exceeding prior skill-evolution methods. We further deploy SkillHone on internal tool-mediated analysis scenarios, where it improves accuracy by an average of 18.8 points across seven settings.

1 Introduction

Skills have emerged as a central mechanism for extending LLM-based agents. A skill is a named, loadable bundle of task-specific procedures, scripts, references, and output conventions that an agent can invoke for a particular class of work (Xu and Yan, 2026; Ling et al., 2026). In deployed agent systems such as Claude Code, Codex, and Hermes, skills serve as a primary unit of specialization. The development of a robust skill is never a one-shot event but an **ongoing process of creation and**

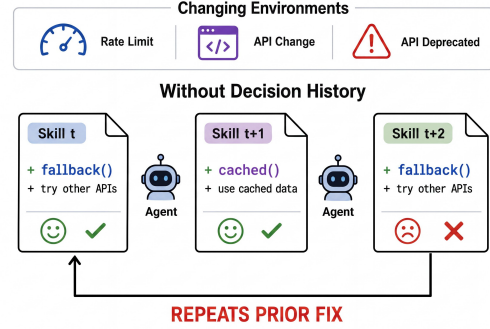


Figure 1: Context loss in artifact-only skill evolution: without decision history, a later agent may repeat a prior optimization and make future improvement harder.

adaptation. Even during its initial formation, a skill requires rigorous debugging and constant updates to broaden its scope or patch unhandled edge cases. As environments shift and APIs change, this creation-maintenance loop persists, making it essential that the rationale behind every revision remains preserved and maintainable throughout its existence. The central methodological question is therefore how later agents can continue improving a skill across many development sessions without losing the rationale behind earlier changes.

Existing methods for improving agent skills can be viewed as two representative routes. Synthesis-style construction methods such as Skill-Creator¹ turn a task description, examples, or supporting documents into a loadable skill and then hand the generated artifact to later sessions. GEPA-style reflective optimization methods, including Hermes-Agent-Self-Evolution (Hermes-SE)², run iterative mutation, evaluation, and selection over skills, prompts, tool descriptions, and code using GEPA (Agrawal et al., 2026). These routes are useful for producing or improving a skill within a

¹<https://github.com/anthropics/skills/blob/main/skills/skill-creator>

²<https://github.com/NousResearch/hermes-agent-self-evolution>

*Work was done when the author was interning at WeChat AI, Tencent Inc., China.

single optimization run, but they largely treat the output as a better artifact.

This artifact-centered view loses the decision context needed for continual maintenance. A later agent may inherit the latest skill, but not the structured history explaining which diagnosis motivated a revision, which alternatives were rejected, what evaluation evidence was observed, and why an outcome was accepted. The same problem applies to the evaluation evidence that supports skill improvement: when later agents rely on probes, validators, traces, or redacted reports, they also need to know which failure mode the evidence reflects and how it shaped prior decisions. As illustrated in Figure 1, rate limits, API changes, and deprecations can arrive over time, and a later optimization step may repeat a previously successful repair that no longer matches the current failure mode. Persistent decision history lets later optimization recover why each revision was made and avoid reintroducing obsolete fixes. Without this context, a locally plausible repair can repeat old edits or make later optimization harder.

We introduce SkillHone, an agent-facing harness for continual skill evolution in skill-enabled agent frameworks. SkillHone maintains the evolving skill together with evaluation assets that support its improvement, and treats skill revisions and evaluation evidence as decision-bearing. For skill revisions, the history records the diagnosis, proposed revision, evaluation evidence, and outcome. For evaluation evidence, it records which probes or reports were used, which failure mode they exposed, and how the evidence justified the outcome. Thus, later agents inherit the skill, evidence, and rationale behind prior changes.

This history is accumulated from many local decisions made during diagnosis, revision, and evaluation. SkillHone therefore uses role-bounded sub-agent dispatches, rather than a fixed multi-agent system, to generate these records as the loop runs: it only assumes an agent runtime that can launch scoped dispatches with separated permissions. This makes the harness portable across agent runtimes such as Claude Code, Codex, and Hermes, while keeping optimization separated from unredacted evaluation assets. Evaluation subagents can inspect probes and traces and run candidate skills, but return only redacted evidence to the optimization side. Optimization subagents use this evidence and prior decision records to form diagnoses and draft skill revisions, without accessing hidden probe targets

or validators. Across sessions, the linked history of diagnoses, revisions, evidence, and outcomes allows later agents to continue skill evolution without re-deriving prior decisions.

We evaluate SkillHone on deep-research benchmarks in the raw open-web setting, where agents must organize retrieval without a pre-integrated search stack. Against skill-evolution baselines and a deep-research agent with commercial retrieval, SkillHone obtains the strongest average results on GAIA and WebWalkerQA-EN under a fixed evaluation-time backbone; the resulting skill bundle also transfers to a different execution backbone (Claude Sonnet 4.6) without additional optimization, indicating that the gain reflects the skill procedure rather than fitting to one model. We also deploy SkillHone on recurring internal tool-mediated analysis scenarios, where it improves the underlying tool-use skills.

Contributions.

1. We identify a gap between artifact-centered skill improvement and continual skill maintenance. Long-lived skills require persistent decision history, rather than a single optimized artifact, so later agents can understand skill revisions together with the evaluation evidence that supported them.
2. We introduce SkillHone, a general agent-facing harness for continual agent skill evolution. It records diagnoses, revisions, evidence, and outcomes under separated roles.
3. We show that SkillHone improves skill development across public benchmarks and deployments.

2 Method

2.1 Problem Setup

In this paper, a skill is a loadable artifact that packages the instructions, procedures, references, and output conventions an agent uses for a class of tasks. SkillHone keeps the model backend and external tools fixed; its goal is to help agents maintain and improve this skill artifact across repeated development sessions.

SkillHone maintains two linked repositories. The skill repository stores the skill bundle being revised. The skill-evaluation repository stores practice probes, oracle targets, validators, traces, redacted reports, and other evaluation assets that provide feedback for skill improvement. The opti-

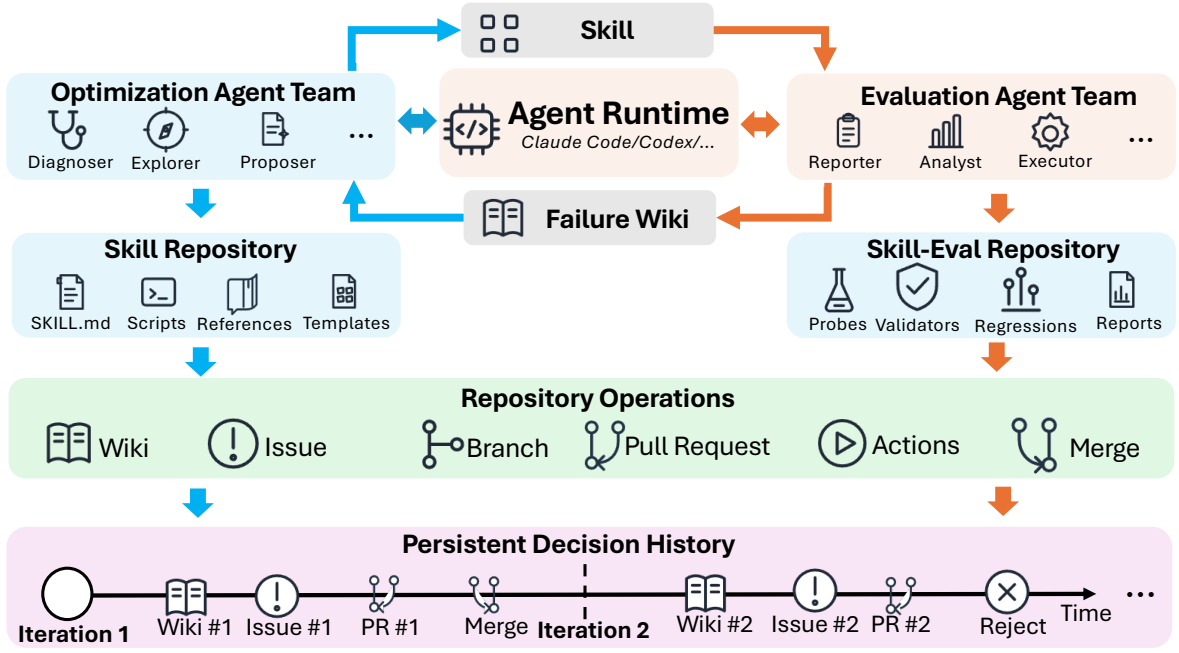


Figure 2: SkillHone architecture. At each development step, an agent runtime dynamically dispatches fresh, role-bounded optimization and evaluation subagents on demand. Optimization dispatches maintain the skill repository and propose diagnosis-guided revisions. Evaluation dispatches run candidate skills against the skill-evaluation repository and export redacted problem reports, without exposing probe targets, validators, or traces. Routed evidence, repository operations, and recorded outcomes form a persistent decision history that later agents reuse for continual skill evolution.

mization side observes only redacted reports, not unredacted targets, validators, or traces. Accepted revisions produce a sequence of skills $S_0, \dots, S_T$, guided by practice feedback and the persistent decision history that links diagnoses, candidate revisions, evaluations, and outcomes.

2.2 SkillHone Harness

As shown in Figure 2, SkillHone separates each development step into optimization and evaluation dispatches. These dispatches are role-bounded so that diagnoses, revisions, redacted evidence, and outcomes can be recorded as auditable decision history. Appendix E (Table 5) lists the recurring dispatch patterns and their permission boundaries.

We formalize the harness as

$$\mathcal{M} = (\mathcal{T}_{\text{opt}}, \mathcal{T}_{\text{eval}}, \mathcal{D}),$$

where $\mathcal{T}_{\text{opt}}$ is the Optimization Agent Team, $\mathcal{T}_{\text{eval}}$ is the Evaluation Agent Team, and $\mathcal{D}$ is the runtime dispatcher.

The optimization side maintains the skill repository, which contains the current `SKILL.md`, scripts, references, templates, and other skill assets. Optimization dispatches can inspect the current skill, read prior decision records and failure summaries,

form diagnoses, propose typed revisions, review candidate changes, and record accepted outcomes. Typical dispatches include diagnosing failures from redacted reports, integrating prior resources, proposing scoped revisions, and drafting patches to S_t .

The evaluation side operates on the skill-evaluation repository, which contains probes, validation suites, regression sets, traces, and evaluation reports. Evaluation dispatches can inspect oracle targets and traces, run the current skill on probe items, analyze failures, and produce redacted reports. They cannot write to the skill repository. Conversely, the optimization side can change the skill but cannot access unredacted probe targets, validators, or execution traces. This separation prevents practice feedback from becoming a direct memorization target.

The dispatcher is a message router rather than a decision authority. At each step, it routes redacted evidence and review requests between the two sides and records accepted outcomes. Acceptance decisions are made using the evidence available to the appropriate side, keeping the decision history tied to the same permission boundary that produced it.

Algorithm 1 SkillHone

Require: Initial skill S_0 , skill repo $\mathcal{R}_s$, eval repo $\mathcal{R}_e$

```

1:  $\mathcal{H}_0 \leftarrow \emptyset$ 
2: for  $t = 0, \dots, T - 1$  do
3:   Eval dispatches run  $A(\cdot; S_t)$  on  $\mathcal{R}_e$  and export  $\tilde{E}_t$ 
4:   Opt dispatches retrieve  $\mathcal{H}_t$ , then propose  $(q_t, r_t)$ 
5:   Eval dispatches return redacted evidence  $e_t$  for  $r_t$ 
6:   Decide  $o_t \in \{\text{ACCEPT}, \text{REVISE}, \text{REJECT}, \text{DEFER}\}$ 
7:    $S_{t+1} \leftarrow \text{APPLY}(S_t, r_t)$  if accepted, else  $S_t$ 
8:    $\mathcal{H}_{t+1} \leftarrow \mathcal{H}_t \cup \{(q_t, r_t, e_t, o_t)\}$ 
9: end for
10: return  $S_T, \mathcal{H}_T$ 

```

2.3 Persistent Decision History

A central design goal of SkillHone is to make skill evolution persistent across sessions. A later agent should not receive only the latest skill snapshot. It should also inherit the structured record of why the skill changed, which alternatives were considered, how candidates were evaluated, and which outcomes were accepted or rejected.

SkillHone represents each development step as a decision record

$$h_t = (q_t, r_t, e_t, o_t),$$

where q_t is the diagnosis, r_t is the candidate revision, e_t is the redacted evaluation evidence, and o_t is the outcome. The outcome may accept the revision, reject it, request further changes, or defer the diagnosis. The decision history up to iteration t is $\mathcal{H}_t = \{h_1, \dots, h_t\}$.

The decision record is more than a version diff. A diff shows how files changed; a decision record links that change to the problem it targeted, the evidence used to evaluate it, and the final decision. This distinction is important for continual evolution. When the environment changes, later subagents can inspect $\mathcal{H}_t$ to determine whether a failure is new, whether a similar fix was already attempted, and why a previous alternative was rejected.

Given a new redacted report $\tilde{E}_t$, optimization subagents retrieve relevant prior records from $\mathcal{H}_{<t}$. They may revisit an old diagnosis, revise a rejected patch under new evidence, or avoid a redundant edit that has already failed. Thus the decision history acts as a persistent memory for the evolution process itself. It enables agents to audit prior changes and continue skill improvement without re-deriving the same diagnoses.

3 Experiments

We evaluate SkillHone on public deep-research benchmarks against existing skill construction and

optimization methods under a fixed execution-time agent backbone, examine cross-backbone transfer of the resulting skill bundle, and report a deployment study on recurring internal tool-mediated analysis scenarios.

3.1 Experimental Setup

We instantiate this comparison in deep research because information seeking is one of the most broadly used agent workloads and naturally benefits from reusable skills: tasks require agents to combine search, navigation, extraction, verification, and recovery across changing web sources, so external resources are rate-limited, fail intermittently, and change format, requiring agents to diagnose failures, revise skills, and preserve the evaluation evidence behind those revisions.

We use two open-domain benchmarks: the text-only validation subset of GAIA (Mialon et al., 2024) and the English split of WebWalkerQA (Wu et al., 2025). GAIA emphasizes multi-step information seeking and factual grounding across heterogeneous sources, while WebWalkerQA-EN focuses on web navigation and information extraction.

We compare two evaluation settings:

- **Curated search:** a deep-research agent uses commercial retrieval services and does not maintain its own search stack.
- **Raw open-web:** agents receive no pre-integrated search tools. They operate through a general agent runtime, public web access, and portable skill bundles that organize search, parsing, extraction, and recovery procedures. Because these skills rely on public pages and free or community search interfaces, they must handle rate limits, unavailable endpoints, and changing page formats without a managed retrieval service.

We separate the controller that optimizes skills from the backbone that executes and evaluates them. Claude Opus 4.6³ serves as the development-time controller that reads feedback, forms diagnoses, and revises skill bundles. Qwen3.6-35B-A3B (Qwen Team, 2026) serves as the execution and evaluation backbone for running skills during development and for producing the final benchmark scores.

To drive skill evolution, we sample 20 items from the publicly released WebShaper dataset (Tao

³<https://www.anthropic.com/news/claude-opus-4-6>

Setting	System	GAIA				WebWalkerQA-EN			
		L1	L2	L3	Avg.	Easy	Med.	Hard	Avg.
Curated search	deep-research agent	61.9	47.0	26.3	48.8	58.5	62.3	67.1	63.2
Raw open-web	Existing-Skills	64.3	33.3	21.1	41.7	51.2	48.5	52.6	50.2
	Skill-Creator	64.3	37.9	21.1	44.1	36.6	36.9	40.8	38.1
	Hermes-SE	73.8	40.9	31.6	50.4	53.7	51.5	55.3	53.0
	SkillHone (Ours)	76.2	66.7	31.6	64.6	53.7	69.2	68.4	66.4

Table 1: Main results on GAIA and WebWalkerQA-EN. Bold marks the best score in each column.

et al., 2025), which is disjoint from GAIA and WebWalkerQA-EN, and adapt them to our agent execution environment. These items serve as the shared development data for all skill-evolution baselines, with each method using them according to its own optimization procedure.

3.2 Baselines

The raw open-web comparison starts from a shared existing-skill pool. We construct this pool from search-related community skills selected by an agent from ClawHub and SkillHub; Appendix A lists the selected skills. The pool is provided to all comparable raw open-web systems for a fair comparison; the systems differ in whether and how they adapt these starting skills.

The raw open-web comparison contains one direct-use setting and three skill-development systems:

- **Existing-Skills:** directly loads the skills as a portable skill bundle, without further optimization.
- **Skill-Creator:** applies built-in iterative skill optimization.
- **Hermes-SE:** applies reflective skill optimization.
- **SkillHone:** runs the full harness with role-separated subagents, redacted evaluation feedback, and persistent decision history.

All raw open-web systems produce the same portable skill-bundle interface: a `SKILL.md` entry file with optional scripts, references, templates, and supporting resources. They differ only in how the bundle is developed.

3.3 Main Results

Table 1 reports the final benchmark results. SkillHone achieves the best average score on both benchmarks, reaching 64.6% on GAIA and 66.4%

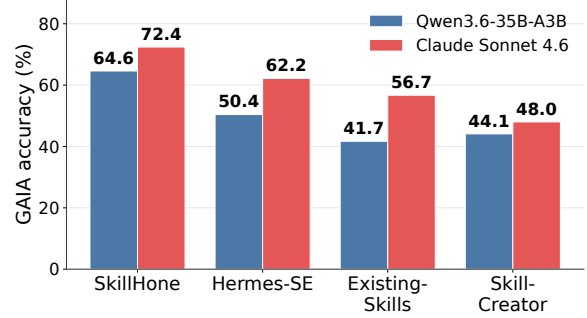


Figure 3: GAIA accuracy under the development backbone (Qwen3.6-35B-A3B) and the transfer backbone (Claude Sonnet 4.6). Each skill bundle is run directly on Claude Sonnet 4.6 without additional optimization.

on WebWalkerQA-EN, 15.8/3.2 points above the deep-research agent despite not using pre-integrated search tools at evaluation time. The gains concentrate on harder splits that require deciding how to search, extracting evidence, verifying findings, and recovering from failed attempts.

Within the raw open-web setting, SkillHone also outperforms the shared-pool skill-development baselines: +20.5/+28.3 points over Skill-Creator and +14.2/+13.4 over Hermes-SE on GAIA/WebWalkerQA-EN. The gap to Existing-Skills shows that exposing the agent to relevant skills is not sufficient: SkillHone turns them into a task procedure for when to search broadly, follow a source, verify an answer, and recover from a failed path, so the main gains come from organization rather than components alone.

Figure 3 further evaluates transfer to Claude Sonnet 4.6⁴: without additional optimization on this backbone, SkillHone reaches 72.4% on GAIA, 10.2/15.7/24.4 points above Hermes-SE/Existing-Skills/Skill-Creator. Appendix C (Figure 4) compares the SkillHone and Hermes-SE optimization trajectories on the same practice probes, illustrating how scoped reverts under persistent decision his-

⁴<https://www.anthropic.com/news/claude-sonnet-4-6>

	GAIA	WebWalkerQA-EN
SkillHone (full)	64.6	66.4
w/o decision history	51.2	55.5
w/o role separation	58.2	61.1

Table 2: Ablation of SkillHone’s two core mechanisms.

tory differ from whole-candidate acceptance under a scalar signal.

3.4 Ablation

To isolate the two core mechanisms, we ablate each under the raw open-web setting and the same Qwen3.6-35B-A3B execution backbone used above. **No decision history** keeps role-separated subagents but starts each step from the latest skill artifact alone, optimizing on a scalar signal as in reflective baselines. **No role separation** keeps decision history but lets a single subagent access the skill repository and unredacted evaluation assets jointly.

Table 2 shows that decision history accounts for the larger share: removing it drops GAIA/WebWalkerQA-EN by 13.4/10.9 points, versus 6.4/5.3 points for role separation.

3.5 Deployment Study

We further evaluate SkillHone on internal tool-mediated analysis scenarios to test whether it improves skills used for recurring analytical requests beyond public web benchmarks. We select several recurring scenarios and their corresponding seed skills from an internal workflow, then apply SkillHone to improve each skill on a fixed exact-match evaluation set per scenario (Appendix F).

Table 3 summarizes the selected scenarios. SkillHone improves six of the seven selected skills over their seeded versions, with the largest gains on counting, aggregation, structure parsing, and span retrieval. The remaining scenario, list filtering, is stable at the aggregate level. Across all listed scenarios, SkillHone improves accuracy by an average of 18.8 points, with the largest gains where the initial procedure leaves reusable analysis steps underspecified.

4 Related Work

Agent skills and skill acquisition. The recent abstraction of agent skills bundles task-specific procedures, scripts, references, and conventions into reusable artifacts that LLM agents can load on demand (Xu and Yan, 2026; Ling et al., 2026).

Scenario	Δ
Counting	+30.0
Aggregation	+26.3
Structure parsing	+25.0
Density estimation	+23.1
Span retrieval	+21.5
Filtered ranking	+5.9
List filtering	+0.0
Avg.	+18.8

Table 3: Deployment study on recurring internal tool-mediated analysis scenarios. Optimized minus seeded accuracy (pp); the final row averages the listed scenarios.

Benchmarks such as Li et al. (2026) and Han et al. (2026) measure when skills help across general tasks and software-engineering settings, while skill-acquisition methods study automated discovery from exploration and feedback (Yang et al., 2025), reinforcement-learning-based evolution (Vishe et al., 2026; Xia et al., 2026), and domain-specific construction (Liu et al., 2026). These works focus on producing or evaluating skills as outputs; SkillHone instead preserves the optimization context—failure modes, candidate revisions, and outcomes—so the next agent inherits the decision history that makes future improvement possible.

Optimization of prompts, systems, and skills.

Within language-system optimization, prompt-only methods use LLMs to propose and select improved instructions (Zhou et al., 2022; Yang et al., 2024a); Reflexion-style agents use verbal feedback to improve behavior across attempts (Shinn et al., 2023); DSPy frames LM pipelines as declarative programs that can be compiled and optimized end-to-end (Khatab et al., 2023). More closely related, GEPA reflectively evolves components of a language system (Agrawal et al., 2026), and Hermes-SE applies this style across skills, prompts, tool descriptions, and code. These methods optimize within a single run over a scalar signal; SkillHone lets later agents continue from where prior runs left off, audit what has been tried, and avoid redundant edits.

Multi-agent collaboration and agent development workflows.

Multi-agent LLM systems show that role specialization improves collaborative problem solving (Talebirad and Nadiri, 2023; Guo et al., 2024; Wang et al., 2024; Tran et al., 2025). MetaGPT organizes agents into specialized

roles for producing software-like artifacts (Hong et al., 2024), and SWE-style benchmarks (Jimenez et al., 2024; Yang et al., 2024b; Xia et al., 2025) study LLM agents in environments where tasks are naturally expressed as diagnosis, code modification, and testing. SkillHone reuses role separation for a different purpose: optimization subagents propose skill revisions while evaluation subagents test candidates and return redacted evidence under a permission-bounded dispatcher, accumulating agent-readable decision history while reducing leakage from evaluation assets to the optimization side.

5 Conclusion

We introduced SkillHone, a harness that evolves skills via role-separated subagents, redacted practice feedback, and persistent decision histories linking problems, revisions, evidence, and outcomes. On GAIA and WebWalkerQA-EN, SkillHone exceeds iterative and reflective skill-development baselines and the deep-research agent, and improves seeded skills by 18.8 points on average across seven recurring internal scenarios. The skill bundle transfers to a different execution backbone without further optimization, suggesting the gain comes from the skill procedure rather than from fitting one model. Decision history lets later agents improve a skill without re-deriving prior decisions.

Limitations

SkillHone currently evolves a single skill in isolation; jointly evolving multiple interdependent skills, including coordination across shared resources and overlapping failure modes, is not addressed in this work.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soyulu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. [GEPA: Reflective prompt evolution can outperform reinforcement learning](#). In *The Fourteenth International Conference on Learning Representations*.
- Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xi-angliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. Swe-skills-bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, and 1 others. 2024. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, and 1 others. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, and 1 others. 2026. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- George Ling, Shanshan Zhong, and Richard Huang. 2026. Agent skills: A data-driven analysis of claude skills for extending large language model functionality. *arXiv preprint arXiv:2602.08004*.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint arXiv:2604.08618*.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pages 9025–9049.
- Qwen Team. 2026. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*.

Zhengwei Tao, Jialong Wu, Wenbiao Yin, Junkai Zhang, Baixuan Li, Haiyang Shen, Kuan Li, Liwen Zhang, Xinyu Wang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. 2025. *WebShaper: Agentically data synthesizing via information-seeking formalization*. Preprint, arXiv:2507.15061.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*.

Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A Rossi, Jingbo Shang, Julian McAuley, and 1 others. 2026. Skill-r1: Agent skill evolution via reinforcement learning. *arXiv preprint arXiv:2605.09359*.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, and 1 others. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.

Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and 1 others. 2025. Web-walker: Benchmarking llms in web traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10290–10305.

Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint arXiv:2511.13646*.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujing Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. In *ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving*.

Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024a. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.

Yongjin Yang, Sinjae Kang, Juyong Lee, Dongjun Lee, Se-Young Yun, and Kimin Lee. 2025. Automated skill discovery for language agents through

exploration and iterative feedback. *arXiv preprint arXiv:2506.04287*.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*.

A Existing Community Skills

The shared existing-skill pool is selected by an agent from search-related community skills on ClawHub and SkillHub:

- web-pilot
- web-content-fetcher
- scholar-search
- query-dbpedia
- multi-search-engine
- literature-review
- geepers-data
- duckduckgo-websearch
- deep-research-pro
- ddgs-search

B Repository Workflow Example

In our implementation, SkillHone realizes persistent decision history through a GitHub-style repository workflow. Issues record diagnosed failure modes, pull requests contain proposed skill revisions, and merge or rejection decisions record the accepted outcome. This is one concrete interface for making the evolution process auditable, rather than a required interface for SkillHone.

Table 4 gives the repository records from one deep-research skill run. The five rows correspond to the five non-seed SkillHone iterations shown in Figure 4. The table highlights that SkillHone optimizes the whole skill bundle, including instructions, scripts, and references, rather than only rewriting a single SKILL.md body as in the Hermes-SE comparison.

C Optimization Trajectory

Figure 4 compares one SkillHone run with one Hermes-SE run on the same practice probes. Both runs start from the same seed skill. SkillHone improves from 30% to 70% while recovering from two regressed revisions through targeted follow-up edits. Hermes-SE instead accepts or skips whole

Iter.	Issues	PRs	Files changed
1	#1 DuckDuckGo search, error handling, and time management	#2 DuckDuckGo search, error handling, and time management	SKILL.md (+29/-13), scripts/ddg_search.sh (+25/-0), scripts/web_search.sh (+25/-7), scripts/wiki_page.sh (+15/-3), scripts/wiki_search.sh (+15/-3), scripts/wiki_section.sh (+39/-9), scripts/wikidata_lookup.sh (+52/-0)
2	#3 real web search, script fixes, write-early strategy	#4 real web search, script fixes, write-early strategy	SKILL.md (+33/-48), scripts/ddg_search.py (+53/-0), scripts/fetch_url_jina.sh (+25/-0), scripts/web_search.sh (+6/-6), scripts/wiki_page.sh (+3/-3), scripts/wiki_search.sh (+3/-3), scripts/wiki_section.sh (+7/-7)
3	#11 merge best of iter1 and iter2 with lighter verification and stricter budget	#12 merge best of iter1 and iter2	SKILL.md (+33/-40)
4	#15 name-format issue and tighter budget	#16 name-format rules and Wikidata SPARQL	SKILL.md (+13/-6), scripts/wikidata_sparql.py (+59/-0)
5	#19 revert budget to 20 while keeping name rules and SPARQL	#21 revert budget to 20, keep name rules and SPARQL	SKILL.md (+45/-46)

Table 4: Repository workflow records for one deep-research skill run. Line counts aggregate the changed files in the PRs listed for each iteration.

prompt candidates under a scalar validation signal. This illustrates the operational value of persistent decision history: later revisions can target the offending part of a change while retaining useful edits.

D Evaluation Repository and Redaction

SkillHone uses a skill-evaluation repository for practice feedback during skill development. The repository stores probe task instances, oracle targets, validators, provenance metadata, and execution traces. Probe items may come from existing sources such as human-written cases, historical failures, or environment-derived checks. It only assumes that evaluation subagents can run the current skill against available probe items and export redacted reports to the optimization side.

For each probe item, we denote the task instance by x_k , the oracle target by y_k , the validator by ϕ_k , and the provenance metadata by π_k . Running the current skill produces

$$m_k = \phi_k(A(x_k; S_t), y_k),$$

where m_k is the validation outcome. The optimization side receives only a redacted problem report

$$\tilde{E}_t = \text{Redact}(\{x_k, y_k, \phi_k, \pi_k, m_k\}_{k=1}^K),$$

which summarizes failure modes, aggregate outcomes, and diagnostic hypotheses without expos-

ing unredacted targets, validators, or traces. Final benchmark evaluation remains outside the optimization loop.

E Subagent Dispatch Patterns and Permissions

The roles in Table 5 are not fixed identities that SkillHone preconfigures in advance. They are recurring dispatch patterns observed when the runtime dispatcher runs the development loop. Each subagent is created on demand, assigned to a permission-bounded team, and granted only the actions needed for that dispatch.

The team boundary is the structural mechanism. Optimization subagents can write to the skill repository but cannot access unredacted probe targets, validators, or traces. Evaluation subagents can inspect probes and traces but cannot write to the skill repository. The runtime dispatcher creates subagents and routes artifacts, but it does not directly edit either repository.

F Deployment Evaluation Protocol

Each scenario in Table 3 is a recurring tool-mediated analysis task. The per-scenario evaluation set consists of representative de-identified requests, with reference answers produced by LLM annotation and verified by human annotators; disagreements are resolved on review. Accuracy is

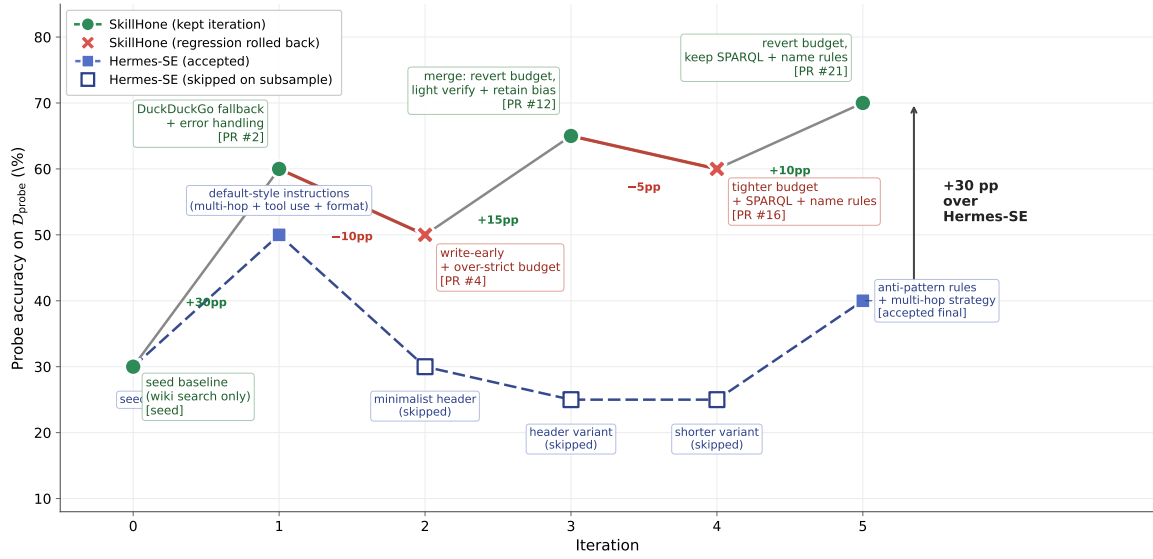


Figure 4: Probe accuracy trajectories for SkillHone and Hermes-SE across five optimization iterations starting from a shared seed skill. SkillHone points are labeled with the accepted revision that introduced each change. When a change regresses performance, the following iteration can target the offending part while preserving useful edits. Hermes-SE optimizes a single SKILL .md body and accepts or rejects whole candidates under a scalar validation signal. The final benchmarks are not available to either system during optimization.

Team	Dispatch pattern	Permission boundary
Optimization	Proposer	Reads redacted reports $\tilde{E}_{<t}$ and decision history $\mathcal{H}_{<t}$; writes diagnoses; no access to probe targets or validators.
Optimization	Explorer	Searches external resources for reusable patterns when a revision needs them; no access to probe targets or validators.
Optimization	Developer	Edits S_t and proposes typed revisions; no access to unredacted evaluation assets.
Optimization	Reviewer	Reviews pending skill changes using redacted evidence and prior decision records; no access to probe targets or validators.
Optimization	Decider	Accepts or rejects candidate revisions according to recorded evidence; no access to unredacted evaluation assets.
Evaluation	Executor	Runs $A(\cdot; S_t)$ on probe items; may inspect oracle targets, validators, and traces; no skill-repository write access.
Evaluation	Diagnoser	Analyzes outcomes and traces; updates evaluation diagnostics; no skill-repository write access.
Evaluation	Reporter	Produces redacted problem reports $\tilde{E}_t$ for the optimization side; no revision-decision authority.
Evaluation	Auditor	Checks probe metadata, traces, and redacted reports for evaluation-side consistency; no skill-repository write access.
Runtime	Dispatcher	Creates subagents and routes artifacts between repositories; no direct skill edits and no access to unredacted probe targets.

Table 5: Recurring subagent dispatch patterns in SkillHone. The role column describes the function performed by a dispatched subagent, while the team column defines its permission boundary. This separation makes the optimizer and evaluator split structural rather than prompt-imposed.

exact-match against the verified reference. The seeded and SkillHone-optimized skills run on the same fixed evaluation set under identical runtime configuration, so the reported Δ reflects skill-level changes rather than evaluation-set drift.